LABORATORIO 6

Visualización de datos con Grafana

Preparado por:
Luisdavid Colina V24766381
Samantha Ramírez V31307714

1. Equipo y herramientas

El laboratorio anterior dejó un broker Mosquitto funcionando y la comprobación de que los mensajes llegaban de una máquina a otra. Lo que faltaba era ver esos datos. Una línea de texto en una terminal sirve para confirmar que el enlace funciona, pero no deja ver una tendencia ni comparar dos momentos. Grafana resuelve esa parte leyendo de una fuente de datos y dibujando la serie en el tiempo [1].

La guía plantea el montaje sobre una Raspberry Pi con un sensor de ultrasonido conectado a sus pines GPIO. El laboratorio no tenía Raspberry Pi disponible el día de la práctica, así que el sensor se montó sobre una placa ESP32 con su tarjeta de expansión, y toda la parte de servidor quedó en el PC de la mesa con Linux Mint. La tabla siguiente resume lo que se usó.

Elemento	**Detalle**
Equipo de trabajo	PC de la mesa, Linux Mint sobre Ubuntu 24.04
Placa	ESP32 con tarjeta de expansión, puerto `/dev/ttyUSB0`
Firmware de la placa	MicroPython `ESP32_GENERIC` v1.23.0
Sensor	Ultrasonido HC-SR04 sobre protoboard
Contenedores	Docker 29.9.0 y Docker Compose v5.6.0
Imágenes	`prom/prometheus:latest` y `grafana/grafana`
Broker	Mosquitto del laboratorio 5, puerto 1883
Herramientas Python	`esptool` y `mpremote` para la placa
	`pyserial` y `paho-mqtt` para el puente

2. Instalación del entorno

2.1. Actualización del sistema y servicio de tiempo

El primer paso fue actualizar los paquetes e instalar `ntpsec`. El servicio de tiempo no es un adorno en este laboratorio. Grafana ubica cada punto de la serie según la marca de tiempo con la que llega, de modo que un reloj desfasado en el equipo desplaza la gráfica completa y hace imposible comparar lo que publican dos máquinas distintas.

Código 1: Actualización del sistema e instalación del servicio de tiempo.

```
sudo apt update && sudo apt upgrade -y
sudo apt-get install -y ntpsec
sudo systemctl status ntpsec --no-pager
```

El servicio quedó activo y sincronizando contra el grupo de servidores públicos (Figura 1).

```
lacore4@lacore4-ThinkCentre-M92p:~$ sudo systemctl status ntpsec --no-pager
● ntpsec.service - Network Time Service
     Loaded: loaded (/usr/lib/systemd/system/ntpsec.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-10-09 09:44:02 -04; 16s ago
       Docs: man:ntpd(8)
    Process: 62454 ExecStart=/usr/libexec/ntpsec/ntp-systemd-wrapper (code=exited, status=0/SUCCESS)
   Main PID: 62457 (ntpd)
      Tasks: 1 (limit: 18602)
     Memory: 10.6M (peak: 11.6M)
        CPU: 44ms
     CGroup: /system.slice/ntpsec.service
             └─62457 /usr/sbin/ntpd -p /run/ntpd.pid -c /etc/ntpsec/ntp.conf -g…

Oct 09 09:44:06 lacore4-ThinkCentre-M92p ntpd[62457]: DNS: dns_check: process…01
Oct 09 09:44:06 lacore4-ThinkCentre-M92p ntpd[62457]: DNS: Pool taking: 51.22…13
Oct 09 09:44:06 lacore4-ThinkCentre-M92p ntpd[62457]: DNS: Pool taking: 41.17…55
Oct 09 09:44:06 lacore4-ThinkCentre-M92p ntpd[62457]: DNS: Pool skipping: 38.…35
Oct 09 09:44:06 lacore4-ThinkCentre-M92p ntpd[62457]: DNS: Pool taking: 193.2…21
Oct 09 09:44:06 lacore4-ThinkCentre-M92p ntpd[62457]: DNS: dns_take_status: 3… 8
Oct 09 09:44:07 lacore4-ThinkCentre-M92p ntpd[62457]: DNS: dns_probe: ntp.ubu…01
Oct 09 09:44:07 lacore4-ThinkCentre-M92p ntpd[62457]: DNS: dns_check: process…01
Oct 09 09:44:07 lacore4-ThinkCentre-M92p ntpd[62457]: DNS: Server taking: 185…57
Oct 09 09:44:07 lacore4-ThinkCentre-M92p ntpd[62457]: DNS: dns_take_status: n… 0
Hint: Some lines were ellipsized, use -l to show in full.
```

Figura 1: Servicio `ntpsec` activo y resolviendo los servidores de tiempo.

2.2. Docker y Docker Compose

Grafana y Prometheus se instalaron como contenedores en lugar de paquetes del sistema. La diferencia práctica es que la imagen trae sus propias dependencias y versiones, así que no hay que pelear con las que ya tiene Linux Mint, y basta borrar el contenedor para dejar el equipo como estaba.

La instalación siguió el repositorio oficial de Docker para Debian. Después de instalar el motor se agregó el usuario al grupo `docker` y se ajustó el permiso sobre el socket, que es lo que permite lanzar contenedores sin anteponer `sudo` a cada comando.

Código 2: Permiso de acceso al socket de Docker.

```
sudo usermod -aG docker $USER
sudo setfacl -m user:$USER:rw /var/run/docker.sock
```

```
Docker version 29.9.0, build f415da8
Docker Compose version v5.6.0
```

Figura 2: Versiones instaladas del motor y del complemento Compose.

2.3. Contenedores de Prometheus y Grafana

Con el motor listo se creó el directorio `contenedor` con los tres archivos de configuración que pide la guía. El `docker-compose.yml` declara los dos servicios, publica el puerto 9090 para Prometheus y el 3000 para Grafana, y enlaza el segundo con el primero. Los *paths* de los volúmenes van absolutos, tal como advierte el enunciado, porque Compose interpreta cualquier ruta relativa como el nombre de un volumen con nombre y no como una carpeta del disco.

La descarga de las dos imágenes tomó algo más de un minuto y medio, y los contenedores quedaron levantados a la primera (Figura 3). Prometheus se instaló por indicación de la guía aunque este laboratorio no lo use, porque queda disponible para las prácticas siguientes.

Código 3: Archivos de configuración y arranque de los contenedores.

```
mkdir -p /home/$USER/contenedor
cd /home/$USER/contenedor
nano docker-compose.yml
nano prometheus.yml
touch grafana.yml
chmod 755 *
docker compose up -d
```

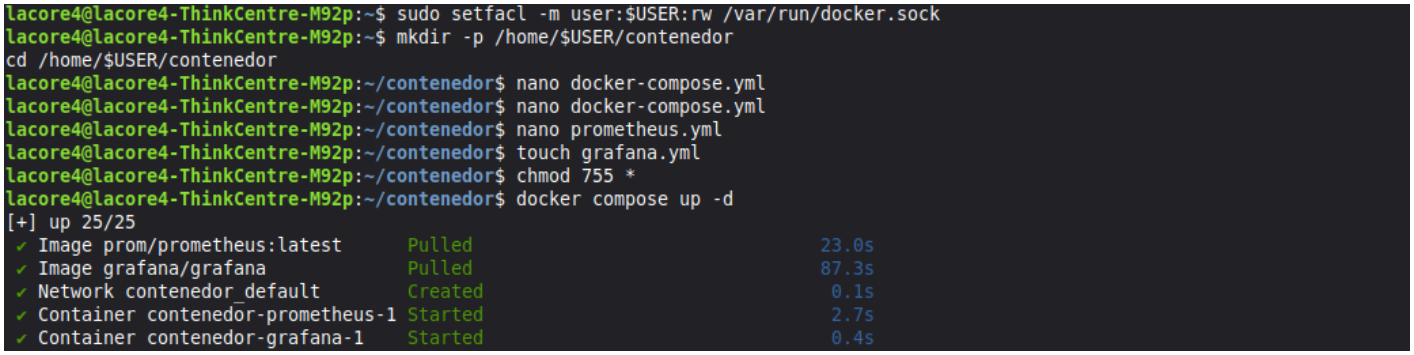


Figura 3: Creación de los archivos y arranque de los dos contenedores.

Ambas interfaces respondieron desde el navegador del mismo equipo. Prometheus quedó en el puerto 9090 con su consola de consultas vacía (Figura 4), y Grafana en el 3000, donde el primer acceso pide cambiar la contraseña `admin` que trae por defecto.

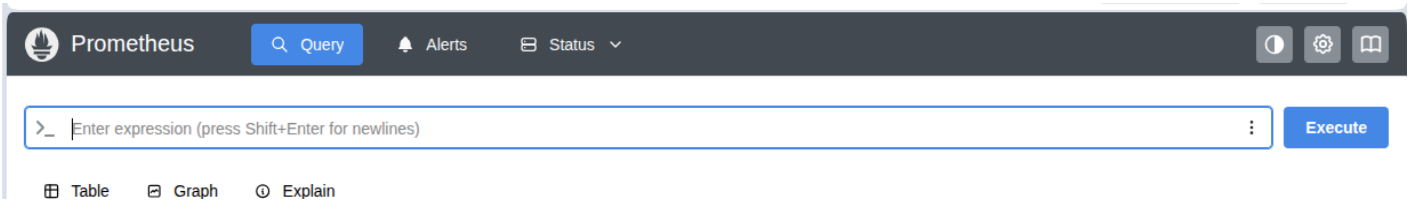


Figura 4: Consola de consultas de Prometheus en el puerto 9090.

3. Adquisición de datos del sensor

3.1. Actividad 5.1: Conexión del sensor y script de recepción

Enunciado: Conecte el sensor de Ultrasonidos a la placa. Programe un script en Python que sea capaz de recibir los datos enviados por el sensor. Muestre el circuito en Fritzing. (5 puntos)

El montaje se hizo sobre hardware real y no en Fritzing, de modo que el circuito queda documentado con la fotografía del armado (Figura 5). El HC-SR04 tiene cuatro terminales. `VCC` y `GND` toman la alimentación de los rieles de la protoboard, `TRIG` recibe el pulso que dispara la medición y `ECHO` devuelve un pulso cuya duración es proporcional al tiempo de vuelo del sonido [4].

Terminal del sensor	Conexión en la placa
`VCC`	Riel de alimentación de la protoboard
`GND`	Riel de tierra de la protoboard
`TRIG`	GPIO 12, configurado como salida
`ECHO`	GPIO 14, configurado como entrada

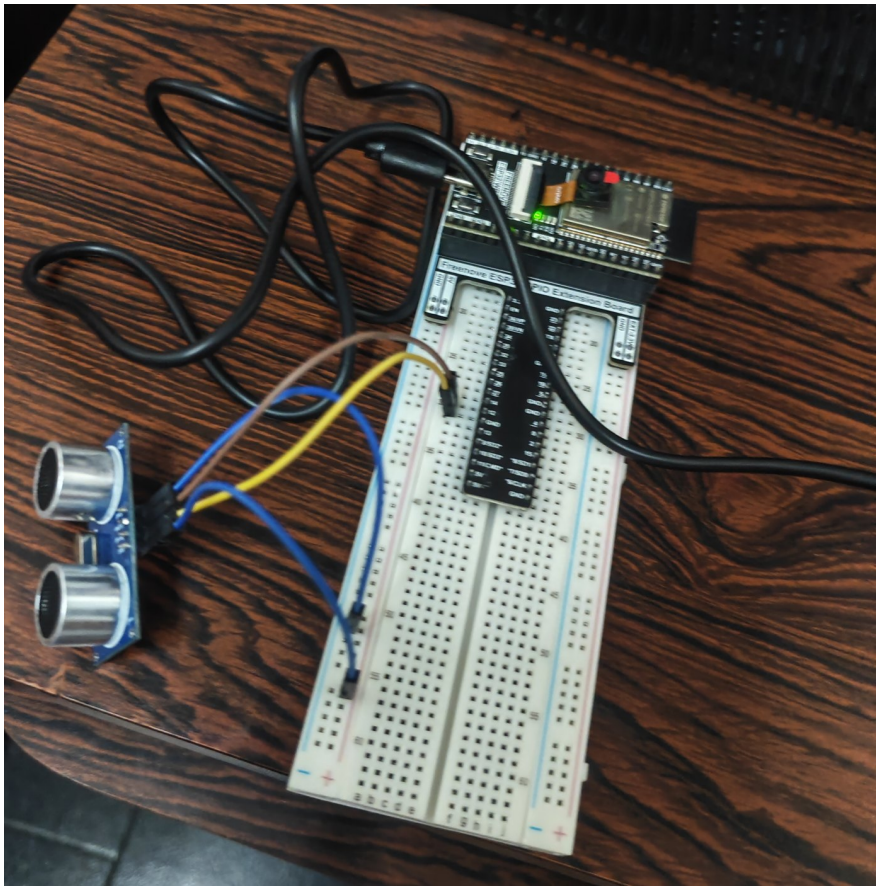


Figura 5: Montaje del HC-SR04 sobre la protoboard junto a la placa ESP32.

Firmware de la placa. La placa venía sin MicroPython, así que primero hubo que grabarlo. El trabajo se hizo desde un entorno virtual de Python con `esptool` para borrar y escribir la memoria flash, y `mpremote` para copiar archivos a la placa ya encendida [3].

```
lacore4@lacore4-ThinkCentre-M92p:~$ python3 -m venv ~/esp_env
~/esp_env/bin/pip install esptool mpremote
~/esp_env/bin/esptool.py --chip esp32 --port /dev/ttyUSB0 erase_flash
~/esp_env/bin/esptool.py --chip esp32 --port /dev/ttyUSB0 write_flash -z 0x1000 ~/ESP32_GENERIC-2
0240602-v1.23.0.bin
```

Figura 6: Entorno virtual, herramientas y grabado del firmware MicroPython.

El borrado previo con `erase_flash` no es opcional. Sin él, los restos del firmware anterior conviven con el nuevo en la misma memoria y la placa arranca en un estado indefinido. La escritura terminó verificando el *hash* de lo grabado y reiniciando la placa por la línea RTS (Figura 7).

```
Hash of data verified.

Hard resetting via RTS pin...
```

Figura 7: Verificación del firmware grabado y reinicio de la placa.

El programa que corre en la placa calcula la distancia a partir del tiempo de vuelo. Pone `TRIG` en alto durante diez microsegundos, mide con `time_pulse_us()` cuánto dura el pulso de `ECHO` y aplica la velocidad del sonido, 343 metros por segundo, dividiendo entre dos porque el sonido recorre el camino de ida y vuelta. El tercer argumento de `time_pulse_us()` es un tiempo de espera de 30 milisegundos que evita que el programa se quede trabado cuando no hay eco de retorno, caso en el que la función devuelve un valor negativo y la medición se reporta como cero.

```
lacore4@lacore4-ThinkCentre-M92p:~$ cat << 'EOF' > main.py
import machine, time

trig = machine.Pin(12, machine.Pin.OUT)
echo = machine.Pin(14, machine.Pin.IN)

def medir():
    trig.value(0)
    time.sleep_us(2)
    trig.value(1)
    time.sleep_us(10)
    trig.value(0)
    t = machine.time_pulse_us(echo, 1, 30000)
    return round((t * 0.0343) / 2, 2) if t > 0 else 0

while True:
    print(medir())
    time.sleep(5)
EOF
~/esp_env/bin/mpremote cp main.py :main.py
~/esp_env/bin/mpremote reset
cp main.py :main.py
```

Figura 8: Programa `main.py` escrito en la placa y copiado con `mpremote`.

El intervalo entre lecturas arrancó en cinco segundos, como se ve en la figura, y después se bajó a medio segundo. Con cinco segundos la gráfica de Grafana quedaba con puntos demasiado separados y el trazo parecía una línea recta entre dos mediciones lejanas.

Recepción desde Python. Del lado del PC, la placa aparece como el archivo de dispositivo `/dev/ttyUSB0`. El script abre ese puerto a 115200 baudios con `pyserial` y lee línea por línea lo que la placa imprime [5]. La conversión a `float` dentro de un `try` es necesaria porque al conectar la placa el puerto entrega además mensajes de arranque de MicroPython, que no son números y harían fallar el programa.

4. Publicación y visualización

4.1. Actividad 5.2: Tópicos en el broker y panel en Grafana

Enunciado: El script debe publicar los datos recibidos como tópicos y enviarlos al broker mosquitto. Siga la siguiente nomenclatura para los tópicos `/NombreGrupo/distancia`, `/NombreGrupo/temperatura`, `/NombreGrupo/humedad`. Visualice sus tópicos en Grafana. (5 puntos)

Prueba previa contra el broker. Antes de conectar la placa se corrió un script que publicaba valores generados en el momento, solo para comprobar que el cliente se autenticaba contra Mosquitto y que los tres tópicos llegaban. Esa prueba dejó claro que la parte de red estaba resuelta, de modo que cualquier falla posterior tenía que venir de la placa o del puerto serial.

```
lacore4@lacore4-ThinkCentre-M92p:~/contenedor$ python3 sensor_mqtt.py
Iniciando publicación de datos... Presiona Ctrl+C para salir.
Enviado -> Distancia: 62.01 | Temp: 21.75 | Humedad: 52.73
Conectado exitosamente al broker MQTT
Enviado -> Distancia: 16.27 | Temp: 26.62 | Humedad: 67.23
Enviado -> Distancia: 21.6 | Temp: 29.83 | Humedad: 56.55
Enviado -> Distancia: 46.52 | Temp: 20.7 | Humedad: 59.02
Enviado -> Distancia: 82.12 | Temp: 24.61 | Humedad: 77.69
Enviado -> Distancia: 12.11 | Temp: 26.24 | Humedad: 43.13
Enviado -> Distancia: 84.17 | Temp: 20.13 | Humedad: 76.33
Enviado -> Distancia: 12.09 | Temp: 26.28 | Humedad: 43.51
Enviado -> Distancia: 89.68 | Temp: 22.51 | Humedad: 44.79
Enviado -> Distancia: 73.48 | Temp: 27.61 | Humedad: 68.29
Enviado -> Distancia: 59.57 | Temp: 29.3 | Humedad: 70.41
Enviado -> Distancia: 89.92 | Temp: 27.91 | Humedad: 45.01
Enviado -> Distancia: 16.06 | Temp: 21.34 | Humedad: 66.31
Enviado -> Distancia: 85.08 | Temp: 22.72 | Humedad: 56.14
Enviado -> Distancia: 67.75 | Temp: 25.02 | Humedad: 63.01
Enviado -> Distancia: 37.13 | Temp: 23.3 | Humedad: 78.87
Enviado -> Distancia: 16.71 | Temp: 23.55 | Humedad: 66.94
Enviado -> Distancia: 15.29 | Temp: 22.18 | Humedad: 71.96
Enviado -> Distancia: 8.52 | Temp: 25.99 | Humedad: 56.01
```

Figura 9: Prueba de publicación con valores generados, antes de conectar la placa.

Puente entre el puerto serial y el broker. El programa definitivo junta las dos mitades. Lee del puerto serial lo que escribe la placa y publica cada lectura en `/Grupo1/distancia` con `paho-mqtt` [6].

Los otros dos tópicos de la nomenclatura llevan un valor fijo de 25,0 y 60,0. El único sensor disponible en la mesa fue el de ultrasonido, que mide distancia y nada más, así que no había de dónde sacar temperatura ni humedad reales. Los tópicos se mantienen publicándose para respetar el esquema de nombres que pide el enunciado, y quedan marcados como simulados en el propio código para que no se confundan con una medición.

```
lacore4@lacore4-ThinkCentre-M92p:~$ cat << 'EOF' > puente.py
import serial
import paho.mqtt.client as mqtt

GRUPO = "Grupo1" # Recuerda cambiar esto en Grafana también
ser = serial.Serial('/dev/ttyUSB0', 115200)
client = mqtt.Client()
client.connect('127.0.0.1', 1883, 60)

print("Escuchando a la placa por USB y enviando a Mosquitto...")

while True:
    if ser.in_waiting > 0:
        linea = ser.readline().decode('utf-8', errors='ignore').strip()
        try:
            d = float(linea)
            client.publish(f"/{GRUPO}/distancia", str(d))
            client.publish(f"/{GRUPO}/temperatura", "25.0") # Simulado
            client.publish(f"/{GRUPO}/humedad", "60.0")     # Simulado
            print(f"MQTT Publicado -> Distancia: {d} cm")
        except ValueError:
            pass # Ignora basura de la consola
EOF
```

Figura 10: Programa `puente.py` que lee el puerto serial y publica al broker.

El primer intento de ejecución falló porque `paho-mqtt` no estaba instalado en el entorno virtual. El error es explícito y se resolvió con una instalación en el mismo entorno, junto con `pyserial` (Figura 11).

```
EOF
~/esp_env/bin/python puente.py
Traceback (most recent call last):
  File "/home/lacore4/puente.py", line 2, in <module>
    import paho.mqtt.client as mqtt
ModuleNotFoundError: No module named 'paho'
lacore4@lacore4-ThinkCentre-M92p:~$ ~/esp_env/bin/pip install pyserial paho-mqtt
```

Figura 11: Falta de la librería `paho` y su instalación en el entorno virtual.

Con la dependencia resuelta, el puente quedó publicando una medición cada medio segundo. La advertencia sobre la versión 1 de la API de *callbacks* que aparece al arrancar no interrumpe nada, solo avisa que `mqtt.Client()` sin argumentos usa la interfaz anterior de la librería.

```
lacore4@lacore4-ThinkCentre-M92p:~$ ~/esp_env/bin/python puente.py
/home/lacore4/puente.py:6: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client()
Escuchando a la placa por USB y enviando a Mosquitto...
MQTT Publicado -> Distancia: 0.0 cm
MQTT Publicado -> Distancia: 63.09 cm
MQTT Publicado -> Distancia: 64.23 cm
MQTT Publicado -> Distancia: 60.99 cm
MQTT Publicado -> Distancia: 3.64 cm
MQTT Publicado -> Distancia: 336.07 cm
MQTT Publicado -> Distancia: 16.36 cm
MQTT Publicado -> Distancia: 43.25 cm
MQTT Publicado -> Distancia: 9.21 cm
MQTT Publicado -> Distancia: 4.92 cm
MQTT Publicado -> Distancia: 3.29 cm
MQTT Publicado -> Distancia: 2.35 cm
MQTT Publicado -> Distancia: 2.33 cm
MQTT Publicado -> Distancia: 2.33 cm
MQTT Publicado -> Distancia: 3.31 cm
MQTT Publicado -> Distancia: 12.62 cm
```

Figura 12: Publicación continua de las lecturas del sensor hacia el broker.

Visualización en Grafana. En Grafana se agregó la fuente de datos MQTT apuntando al broker local y se armó un panel sobre el tópico `/Grupo1/distancia` [2]. La gráfica responde en vivo al movimiento de la mano frente al sensor (Figura 13). El valor señalado en el panel, 27,8 centímetros a las 10:14:26, corresponde a una de las lecturas publicadas por el puente.

Los valores altos aislados que se ven en la serie, por encima de los 300 centímetros, no son un error del montaje. Ocurren cuando el pulso de ultrasonido no encuentra una superficie perpendicular y rebota contra algo lejano antes de volver, de modo que el tiempo de vuelo medido es mayor que el real. Con la mano quieta frente al sensor la serie se mantiene estable.

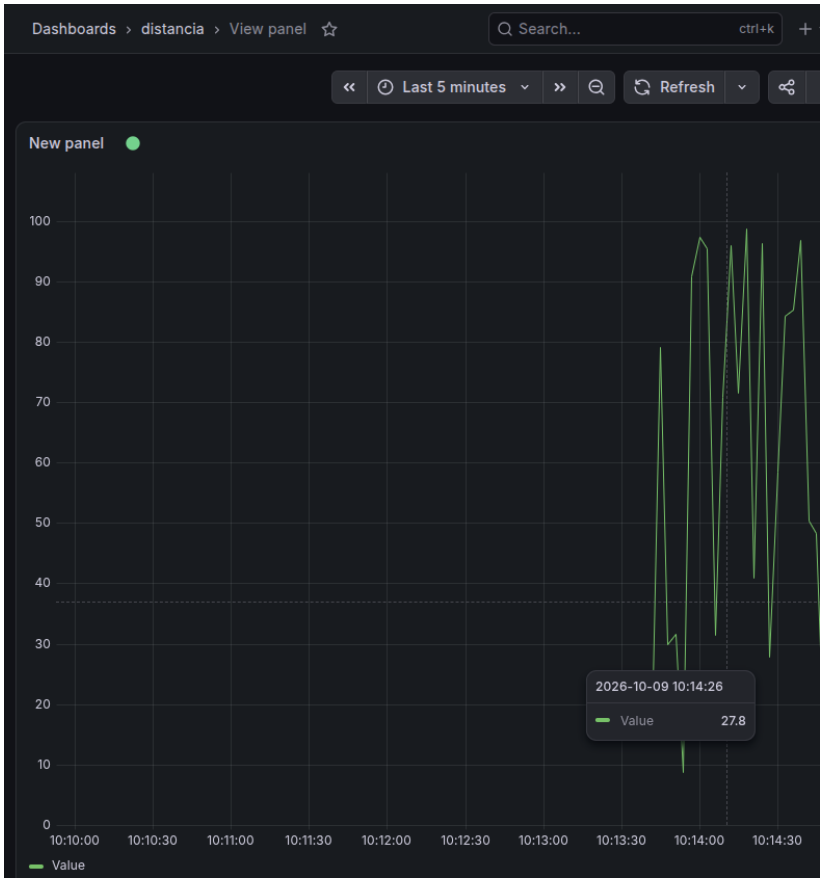


Figura 13: Panel de Grafana con la serie del tópico `/Grupo1/distancia`.

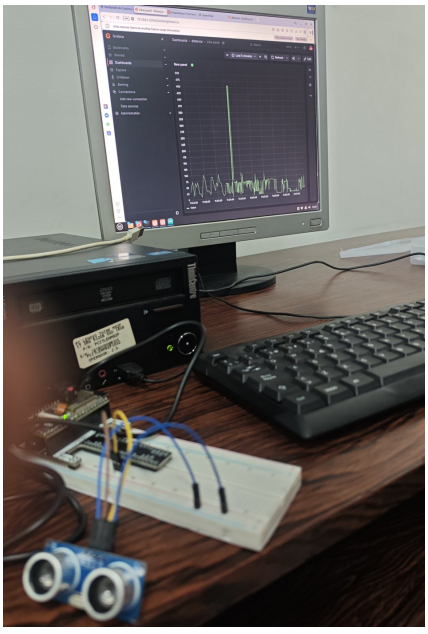


Figura 14: Montaje en funcionamiento con el panel de Grafana actualizándose.

5. Conclusiones

El laboratorio cerró la cadena completa de un sistema IoT básico. El sensor entrega una medición física, el microcontrolador la convierte en un número, un programa en el PC la publica en el broker y Grafana la dibuja. Cada pieza habla con la siguiente por un protocolo distinto, y ninguna necesita saber cómo funcionan las demás.

La pieza que hizo falta construir fue el puente. El ESP32 no habla MQTT por el cable USB, solo escribe texto en el puerto serial, y Grafana no lee puertos seriales. Ese programa de veinte líneas es el que traduce entre los dos mundos, y es también el lugar donde se decide el nombre del tópico, que es lo que después permite distinguir el dato de un grupo del de otro dentro del mismo broker.

Trabajar con contenedores cambió la forma de montar el servidor respecto a los laboratorios anteriores. Mosquitto se instaló con `apt` y quedó mezclado con el resto del sistema, mientras que Prometheus y Grafana se levantaron con un solo comando y se pueden borrar sin dejar rastro. El precio es que los archivos de configuración tienen que ir montados con rutas absolutas desde el disco, porque el contenedor no ve el sistema de archivos del equipo.

El punto que más tiempo consumió no fue ninguno de los conceptos del laboratorio, sino el grabado del firmware en la placa. La memoria flash hay que borrarla antes de escribirla, la escritura toma cerca de dos minutos y el entorno virtual de Python tiene que tener cada librería instalada dentro, no en el sistema. Son pasos mecánicos, pero cualquiera que se salte deja la placa en un estado en el que no responde y no hay un mensaje de error que apunte a la causa.

Queda claro también para qué sirve la visualización. Las mismas lecturas que en la terminal son una lista de números sin forma, en la gráfica muestran de inmediato el rango habitual de la medición y las mediciones sueltas que se salen de ese rango.

Referencias

[1] Grafana Labs, "Run Grafana Docker image," *Grafana Documentation*, 2024. [En línea]. Disponible: `https://grafana.com/docs/grafana/latest/setup-grafana/installation/docker/`. [Consulta: oct. 2026]

[2] Grafana Labs, "MQTT data source for Grafana," *GitHub Repository*, 2024. [En línea]. Disponible: `https://github.com/grafana/mqtt-datasource`. [Consulta: oct. 2026]

[3] MicroPython, "Quick reference for the ESP32," *MicroPython Documentation*, 2024. [En línea]. Disponible: `https://docs.micropython.org/en/latest/esp32/quickref.html`. [Consulta: oct. 2026]

[4] Elec Freaks, "Ultrasonic Ranging Module HC-SR04," hoja de datos técnica, 2024. [En línea]. Disponible: `https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf`. [Consulta: oct. 2026]

[5] C. Liechti, "pySerial Documentation," 2001–2024. [En línea]. Disponible: `https://pyserial.readthedocs.io/en/latest/`. [Consulta: oct. 2026]

[6] Eclipse Foundation, "Eclipse Paho MQTT Python Client," 2024. [En línea]. Disponible: `https://eclipse.dev/paho/index.php?page=clients/python/index.php`. [Consulta: oct. 2026]

[7] A. Russoniello, "Laboratorio: Visualización de Datos con Grafana," guía de práctica de laboratorio, Laboratorio General (Internet de las Cosas), Escuela de Computación, Facultad de Ciencias, Universidad Central de Venezuela, Caracas, 2026.